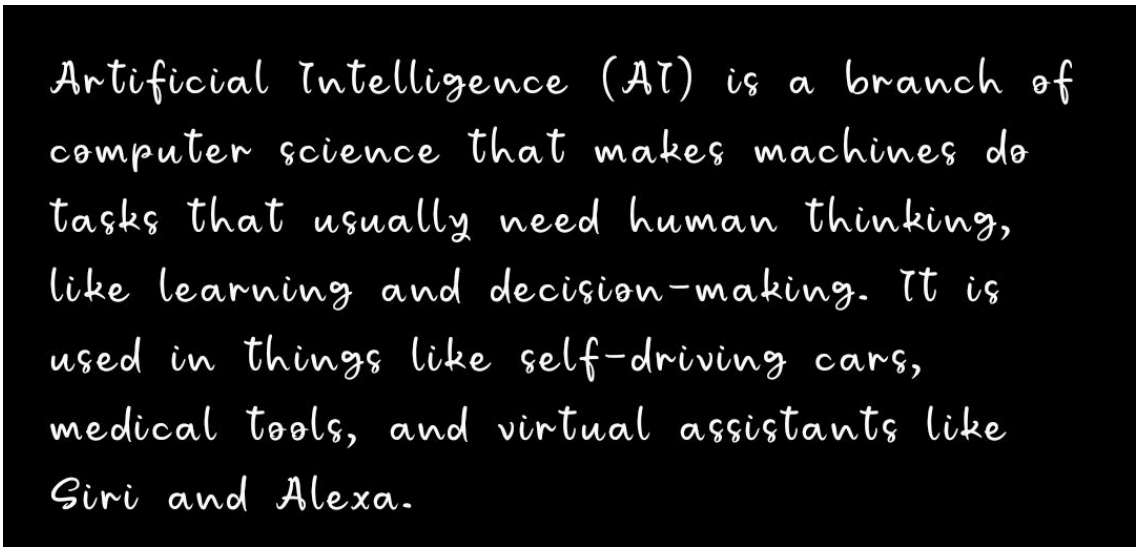
1. Summarization

Initial Prompt:

> Summarize the following paragraph in two simple sentences suitable for a high school student:

“Artificial Intelligence (AI) is a rapidly evolving field of computer science that focuses on creating systems capable of performing tasks that typically require human intelligence. These tasks include reasoning, learning, decision-making, and language understanding. AI is used in various applications such as self-driving cars, medical diagnosis, financial forecasting, and personal digital assistants like Siri and Alexa.”

Initial Output (screenshot):



Refined Prompt:

> Summarize the same paragraph in two simple sentences that a high school student can easily understand.

Refined Output (screenshot):

Artificial Intelligence (AI) is a type of computer technology that helps machines think and learn like humans. It is used in things like self-driving cars, medical checkups, and smart assistants like Siri and Alexa.

Why the refinement is better:

More concise constraint (two sentences)

Explicit target audience (high school student)

Simpler and more focused result

2. Rewriting

Initial Prompt:

> Rewrite the following paragraph in a more formal and professional tone:

“The company is doing great. We’ve added some cool features and our users really love them. We also fixed a bunch of bugs and things are way smoother now.”

Initial Output (screenshot):

The company is performing exceptionally well. We have introduced several valuable new features that have been well received by our users. Additionally, numerous bugs have been resolved, resulting in a significantly smoother and more reliable user experience.

Refined Prompt:

> Rewrite this paragraph in a formal professional tone, using complete sentences and avoiding informal words like “cool” or “stuff.”

Refined Output (screenshot):

The company is experiencing strong performance. We have implemented several innovative features that have been positively received by our users. Furthermore, we have addressed numerous software issues, leading to a more seamless and efficient user experience.

Why the refinement is better:

Clearly avoids colloquial words (“cool,” “bunch”)

Requests complete sentences and formal voice

3. Code Generation

Initial Prompt:

> Generate a Python function to check whether a given number is a prime number or not. The function should return True if the number is prime, otherwise False.

Initial Output (screenshot):

Python Copy code

```python
def is_prime(n):
    """Check if a number is a prime number."""
    if n <= 1:
        return False
    if n == 2:
        return True
    if n % 2 == 0:
        return False

    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False
    return True
```

Refined Prompt:

> Generate a Python function named is_prime(n) which takes an integer n and returns True if it's prime, False otherwise. Include comments and handle edge cases (e.g. n ≤ 1).

Refined Output (screenshot):

Python Copy code

```python
def is_prime(n):
    """
    Check if a number is a prime number.

    Parameters:
    n (int): The number to check.

    Returns:
    bool: True if n is prime, False
otherwise.
    """

    # Edge case: numbers less than or
equal to 1 are not prime
    if n <= 1:
        return False

    # 2 is the only even prime number
    if n == 2:
        return True

    # Even numbers greater than 2 are not
prime
    if n % 2 == 0:
        return False

    # Check for factors from 3 to the
square root of n (inclusive), skipping
even numbers
    for i in range(3, int(n**0.5) + 1, 2):
        if n % i == 0:
            return False

    # If no factors were found, the number
is prime
    return True
```

Why the refinement is better:

Function name specified for clarity

Requests comments & edge-case handling

Ensures more robust and readable code

4. Role-Based Prompt

Initial Prompt:

> You are a Data Scientist. Explain the concept of overfitting in machine learning to a beginner using a real-life example and avoid technical jargon.

Initial Output (screenshot):

Sure! Let's imagine you're helping your friend prepare for a school exam. You both look at last year's question paper and your friend memorizes **every single question and answer** from it—word for word.

Now, on the day of the actual exam, your friend finds that **some questions are different.** Even though they studied hard, they struggle because they **only memorized** last year's paper instead of **understanding the subject.**

This is what **overfitting** looks like in machine learning.

In machine learning, we train a model (like teaching your friend). If the model **memorizes the training data too perfectly,** it performs well on that data but **fails when given new, unseen data**—just like your friend did in the exam.

Real-life analogy summary:

- **Overfitting** = Memorizing instead of understanding.

Refined Prompt:

> As an experienced data scientist, explain overfitting using a simple real-life analogy (like memorizing math problems) to someone new to machine learning. Keep the language non-technical.

Refined Output (screenshot):

Let's say the child practices by solving **10 math problems** at home. Instead of learning the method or pattern, the child **just memorizes the answers** to those 10 questions.

The next day, in school, the teacher gives a **new set of math problems**—they look similar but aren't exactly the same.

Because the child didn't understand **how** to solve the problems and only memorized the old answers, they now **struggle with the new questions.**

That's what overfitting is in machine learning:

- The model (like the child) **memorizes** the training data too well.
- It performs great on that training data (just like the child with the 10 known problems).
- But when given new data, it **fails to**

Why the refinement is better:

Specifies the tone (“experienced data scientist”)

Offers a guiding analogy

Reinforces no technical jargon

5. Structured Content Prompt (JSON/Table)

Initial Prompt:

> Create a JSON object that stores the information of a student including: Name, Age, Roll Number, Subjects (as a list), and Contact details (phone and email as nested values).

Initial Output (screenshot):

```json
{
  "name": "Srikanth Orisi",
  "age": 21,
  "roll_number": "23DS1001",
  "subjects": ["Mathematics", "Data Structures", "Machine Learning", "Operating Systems"],
  "contact_details": {
    "phone": "+91-9876543210",
    "email": "srikanth.orisi@example.com"
  }
}
```

Refined Prompt:

> Generate a JSON object named student with keys:

name (string)

age (number)

roll_number (string or integer)

subjects (array of strings)

contact (object with phone and email as strings)
Include example values for each field.

Refined Output (screenshot):

Sure! Here's a JSON object named student with all the requested keys and example values:

Json Copy code

```
{
  "student": {
    "name": "Anika Sharma",
    "age": 20,
    "roll_number": "CS2025A104",
    "subjects": ["Physics", "Chemistry",
"Computer Science", "Mathematics"],
    "contact": {
      "phone": "+91-9123456789",
      "email": "anika.sharma@example.com"
    }
  }
}
```

Why the refinement is better:

Specifies exact data types and field names

Requests example values for completeness

Improves structured consistency